

A DETAILED GUIDE FOR THE INSTALLATION AND RUNNING OF THE ARCULUS ZERO TRUST DRONE TESTBED AND APPLICATION

I. (Optional) Installing the Community Honey Network (CHN) Server

The CHN Server is an open-source application used to deploy honeypots on any devices on our network. These honeypots can be used to monitor activity from malicious users, develop threat intelligence based on these actions, build attack signatures, and achieve active defense by pretense. While this is an additional plugin that delivers utility to Arculus, its installation is optional, and all other functionalities will work just fine without it.

1. First, on a new Ubuntu device to host the CHN Server, update the APT package library using the command **sudo apt update**. Then install the required packages using the command **sudo apt-get install -y python3-validators docker.io**. Install Docker Compose using the below commands:

```
sudo curl -L
"https://github.com/docker/compose/releases/download/v2.0.1/docker-compose-
$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose
```

```
sudo chmod +x /usr/local/bin/docker-compose
```

```
sudo systemctl restart docker
```

```
ubuntu@ip-172-31-7-86:~$ sudo apt-get update
Hit:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Hit:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease
Get:4 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:5 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [318 kB]
Get:6 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main Translation-en [82.9 kB]
Get:7 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 c-n-f Metadata [5676 B]
Get:8 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [319 kB]
Get:9 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe Translation-en [134 kB]
Get:10 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 c-n-f Metadata [12.6 kB]
Get:11 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [265 kB]
Get:12 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [63.3 kB]
Get:13 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadata [3668 B]
Get:14 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [247 kB]
Get:15 http://security.ubuntu.com/ubuntu noble-security/universe Translation-en [107 kB]
Get:16 http://security.ubuntu.com/ubuntu noble-security/universe amd64 c-n-f Metadata [9220 B]
Fetched 1819 kB in 1s (1789 kB/s)
Reading package lists... Done
```

```
ubuntu@ip-172-31-7-86:~$ sudo apt-get install -y python3-validators docker.io
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
docker.io is already the newest version (24.0.7-0ubuntu4).
The following NEW packages will be installed:
  python3-decorator python3-validators
0 upgraded, 2 newly installed, 0 to remove and 21 not upgraded.
Need to get 25.3 kB of archives.
After this operation, 114 kB of additional disk space will be used.
Get:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/main amd64 python3-decorator all 5.1.1-5 [10.1 kB]
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 python3-validators all 0.20.0-2 [15.2 kB]
Fetched 25.3 kB in 0s (709 kB/s)
Selecting previously unselected package python3-decorator.
(Reading database ... 106389 files and directories currently installed.)
Preparing to unpack .../python3-decorator_5.1.1-5_all.deb ...
Unpacking python3-decorator (5.1.1-5) ...
Selecting previously unselected package python3-validators.
Preparing to unpack .../python3-validators_0.20.0-2_all.deb ...
Unpacking python3-validators (0.20.0-2) ...
Setting up python3-decorator (5.1.1-5) ...
Setting up python3-validators (0.20.0-2) ...
Scanning processes...
Scanning candidates...
Scanning linux images...
```

2. Clone the CHN Server git repository and enter the root directory of the code.

```
ubuntu@ip-172-31-21-212:~$ sudo mkdir -p /opt && sudo git clone https://
Cloning into '/opt/chnserver'...
remote: Enumerating objects: 221, done.
remote: Counting objects: 100% (99/99), done.
remote: Compressing objects: 100% (69/69), done.
remote: Total 221 (delta 61), reused 63 (delta 29), pack-reused 122
Receiving objects: 100% (221/221), 58.03 KiB | 6.45 MiB/s, done.
Resolving deltas: 100% (130/130), done.
ubuntu@ip-172-31-21-212:~$ cd /opt/chnserver
ubuntu@ip-172-31-21-212:/opt/chnserver$
```

3. Configure the CHN Server using the command `./guided_docker_compose.py`, supply it with the public domain name of the host machine and set all other options as shown below.

```
ubuntu@ip-172-31-21-212:/opt/chnserver$ sudo ./guided_docker_compose.py
Please enter the URL where you'd like your CHN web console available. Note that the domain must be resolvable
Domain: ec2-54-221-98-176.compute-1.amazonaws.com
Please enter a Certificate Strategy. This should be one of:

CERTBOT: Signed certificate by an ACME provider such as LetsEncrypt. Most folks will want to use this. You
tion here
BYO: Bring Your Own. Use this if you already have a signed cert, or if you want a real certificate without
SEKFSIGNED: Generate a simple self-signed certificate
Certificate Strategy: SEKFSIGNED
Wrote file to config/sysconfig/chnserver.env

How many days of honeypot data should be maintained in the database (default 30 days)?
Number of Days: 30
Wrote file to config/sysconfig/mnemosyne.env
Do you wish to enable logging to a remote CIFv3 server? [y/N]: N
Do you wish to enable logging to a local file? [y/N]: y

splunk: Comma delimited key/value logging format for use with Splunk
json: JSON formatted log format
arcsight: Log format for use with ArcSight SIEM appliances
json_raw: Raw JSON output from hpfeeds. More verbose than other formats, but also not normalized. Can generate
Logging Format: json
Wrote file to config/sysconfig/hpfeeds-logger.env
Do you wish to enable intelligence feeds from a remote CIF instance? [y/N]: N
ubuntu@ip-172-31-21-212:/opt/chnserver$
```

4. Quickstart the CHN Server on Docker Compose using the command `sudo docker-compose up -d`, and fetch the user credentials using the command `grep SUPERUSER /opt/chnserver/config/sysconfig/chnserver.env`

```
ubuntu@ip-172-31-21-212:/opt/chnserver$ sudo docker-compose up -d
[+] Running 6/6
  # Container chnserver-redis-1      Started
  # Container chnserver-mongodb-1   Started
  # Container chnserver-hpfeeds3-1   Started
  # Container chnserver-chnserver-1  Started
  # Container chnserver-mnemosyne-1  Started
  # Container chnserver-hpfeeds-logger-1 Started
ubuntu@ip-172-31-21-212:/opt/chnserver$ grep SUPERUSER /opt/chnserver/config/sysconfig/chnserver.env
SUPERUSER_EMAIL=admin@localhost
SUPERUSER_PASSWORD=5j1VQowhdpEf7f8iLUNddw7FPzY014Fy
ubuntu@ip-172-31-21-212:/opt/chnserver$
```

5. Access the CHN Server web application using the public domain of the host machine, and login using the credentials fetched in the previous step.

Welcome to the Community
Honeypot Network Server

Log In

Email

Password

[Forgot password?](#)

6. Once logged in, go to the settings page of the application and note down the API key of the CHN Server. This will be used in the future setup of the Arculus application.



Your Info

Email: admin@localhost

Apikey: 0fa389392b1c44c7923e4d28365d6ca3

Users

admin@localhost (me)

Add User

ADD

Change Password

CHANGE

- Click the Deploy tab on the top bar and select script for any honeypot type. A script will be generated containing an 8-letter string as shown in the screenshot below. This is the deploy key of the CHN Server. This will allow authorized deployment of honeypots through Arculus while utilizing the capabilities of the CHN Server.

CHN Server

Deploy

Attacks

Payloads

Sensors

Charts

Settings

LOGOUT

Select Script

Architecture

Default - ssh-auth-logger

Intel

Deploy Command

```
sudo wget "https://ec2-54-221-98-176.compute-1.amazonaws.com/api/script/?text=true&script_id=8" -O deploy.sh
&& sudo bash deploy.sh https://ec2-54-221-98-176.compute-1.amazonaws.com/6NcFJBem && sudo docker-compose up -d
```

Deploy Script

Name

Default - ssh-auth-logger

Script

#!/bin/bash

II. Setting up the Arculus testbed and application

- On a new Ubuntu host machine, configure the below inbound firewall rules to allow effective communication among the key components of the Arculus testbed and application.

	Transport-layer Protocol	Ports	Description
HTTPS	TCP	443	HTTPS communication
HTTP	TCP	80	HTTP communication
SSH	TCP	22	Remote SSH access
TCP	TCP	179	K3s worker nodes communication access
TCP	TCP	3000 – 3003	Web application UI and API access
TCP	TCP	5000	Honeypot deployment APIs
TCP	TCP	6006	Documentation pages
TCP	TCP	10250	Kubernetes master node communication
UDP	UDP	8285, 8472	K3s worker nodes communication access

Here is an implementation of these firewall settings on AWS EC2 resources configured using security group rules.

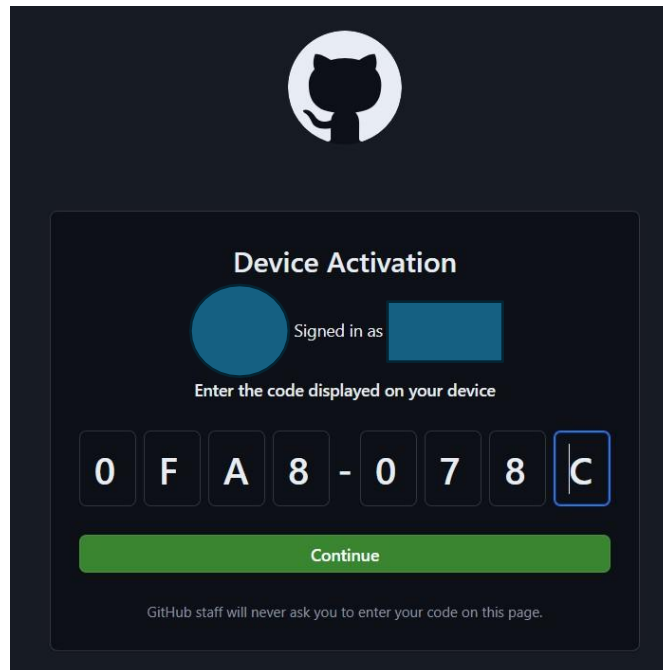
Inbound rules Info					
Type Info	Protocol Info	Port range Info	Source Info		Description - optional Info
HTTPS	TCP	443	Custom	Q	Delete
				0.0.0.0/0 X	
HTTP	TCP	80	Custom	Q	Delete
				0.0.0.0/0 X	
Custom UDP	UDP	8285	Custom	Q	kubernetes communication Delete
				0.0.0.0/0 X	
SSH	TCP	22	Custom	Q	Delete
				0.0.0.0/0 X	
Custom TCP	TCP	6006	Custom	Q	documentation pages Delete
				0.0.0.0/0 X	
Custom TCP	TCP	179	Custom	Q	Worker nodes comm Delete
				0.0.0.0/0 X	
Custom TCP	TCP	5000	Custom	Q	Honeypots Delete
				0.0.0.0/0 X	
Custom TCP	TCP	10250	Custom	Q	Master node Delete
				0.0.0.0/0 X	
Custom UDP	UDP	8472	Custom	Q	kubernetes worker communications Delete
				0.0.0.0/0 X	
Custom TCP	TCP	3000 - 3003	Custom	Q	applications ports Delete
				0.0.0.0/0 X	

2. Install “gh”, a GitHub utility for easy authentication to be able to clone the private Arculus repository as an authorized user. Then, using the command **gh auth login**, authenticate the user through a browser using a one-time code.

```
ubuntu@ip-172-31-26-207:~$ sudo apt install gh
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  gh
0 upgraded, 1 newly installed, 0 to remove and 26 n
Need to get 8826 kB of archives.
After this operation, 45.4 MB of additional disk sp
Get:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu
Fetched 8826 kB in 0s (32.3 MB/s)
Selecting previously unselected package gh.
(Reading database ... 67739 files and directories c
Preparing to unpack ../gh_2.45.0-1ubuntu0.1_amd64.
Unpacking gh (2.45.0-1ubuntu0.1) ...
Setting up gh (2.45.0-1ubuntu0.1) ...
Processing triggers for man-db (2.12.0-4build2) ...
Scanning processes...
Scanning linux images...
```

```
ubuntu@ip-172-31-26-207:~$ gh auth login
? What account do you want to log into? GitHub.com
? What is your preferred protocol for Git operations on this host? HTTPS
? Authenticate Git with your GitHub credentials? Yes
? How would you like to authenticate GitHub CLI? Login with a web browser

! First copy your one-time code: 0FA8-078C
Press Enter to open github.com in your browser...
! Failed opening a web browser at https://github.com/login/device
exec: "xdg-open,x-www-browser,www-browser,wslview": executable file not
Please try entering the URL in your browser manually
```



3. Clone the GitHub repository as an authorized user and enter the root directory of the codebase.

```
ubuntu@ip-172-31-26-207:~$ git clone https://github.com/mizzouceri/Arculus.git
Cloning into 'Arculus'...
remote: Enumerating objects: 688, done.
remote: Counting objects: 100% (688/688), done.
remote: Compressing objects: 100% (597/597), done.
remote: Total 688 (delta 76), reused 688 (delta 76), pack-reused 0
Receiving objects: 100% (688/688), 36.51 MiB | 48.68 MiB/s, done.
Resolving deltas: 100% (76/76), done.
ubuntu@ip-172-31-26-207:~$ cd Arculus/
ubuntu@ip-172-31-26-207:~/Arculus$
```

4. Setup the execute permissions of the automated application script using the command **sudo chmod +x arculus-setup.sh** and execute the script using the command **./arculus-setup.sh**

```
ubuntu@ip-172-31-26-207:~/Arculus$ sudo chmod +x arculus-setup.sh
./arculus-setup.sh
Starting setup process...
Running initial setup script...
Hit:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease
```

5. Furnish configuration information as prompted. First, provide a password to protect the Arculus MySQL database. This will be set to the database, as well as securely shared with the application to make the database accessible to the backend.

```
[INFO] systemd: Starting K3S
Enter database password:
cericenter
Installing Docker and NPM...
Hit:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu noble-security InRelease
Reading package lists... Done
```

6. Use a Random Key Generator tool to generate a 256-bit encryption key and supply it to the application. This is the secret that will be used for secure signing of the JWT authentication tokens. Further, provide the details (Server URL, API key and deploy key) of the CHN Server for letting Arculus access the capabilities of the CHN Server.

```

Starting MySQL container in detached mode...
eed2479125fb4545c020d26124c37b1f8c7205d68a1259892e12cc48db906da3
Enter a 256-bit encryption key:
ICKEGP6S0uhdFcXZfSatHSqRyvks8oF
Enter CHN Server URL (leave blank if not available):
ec2-44-193-76-143.compute-1.amazonaws.com
Enter CHN API key (leave blank if not available):
d568c953fe7c41b6b5549b2b3e105230
Enter CHN deploy key (leave blank if not available):
LEYMtqP9

```

- After all the configuration steps, the Node.js backend and the React.js UI will be deployed, and the application will be accessible at **`http://{arculusHostMachinePublicIPAddress}/3000`**

```

[PM2] Spawning PM2 daemon with pm2_home=/home/ubuntu/.pm2
[PM2] PM2 Successfully daemonized
[PM2] Starting /home/ubuntu/Arculus/arculus-gcs-node/index.js in fork_mode (1 instance)
[PM2] Done.

```

id	name	namespace	version	mode	pid	uptime	U	status
0	node-app	default	1.0.0	fork	13320	0s	0	online

```

Starting UI server with PM2...
[PM2] Starting /home/ubuntu/Arculus/arculus-gcs-ui/server.js in fork_mode (1 instance)
[PM2] Done.

```

id	name	namespace	version	mode	pid	uptime	U	status
0	node-app	default	1.0.0	fork	13320	2m	0	online
1	ui-server	default	0.1.0	fork	17925	0s	0	online

Arculus successfully setup! Access the Arculus UI at `http://52.23.225.56:3000`

III. Navigating through the User and Device Management Dashboards of the Arculus Application

- On visiting the UI's web URL for the first time before creating any users, the application takes us to a "Getting Started" page where users will be prompted to create the first ever user, also a super user. Once the first user is created, this page will be non-functional and only insiders with "Mission Creator" privileges will be able to add new users to the application.



Welcome to Arculus! Get Started!

Username *

firsttimeuser

Email ID *

firsttimeuser@missouri.edu

Role

Mission Creator

Domains

Explosive Ordnance Reconnaissance

Covert Operations Support

Structural Integrity Assessment

Crisis Response and Restoration

Select ...

Password *

.....

Re-enter Password *

.....

CREATE USER



Welcome to Arculus! Get Started!



User Created!
Redirecting...

- After the user is created, the page redirects to the login page, where the first ever user or subsequently created users can log in using their credentials.



Sign in

Username *
firsttimeuser

Password *

☐ Remember me

SIGN IN

EECS Dept., University of Missouri 2024.

3. On successful password authentication, the user is taken to the “About” page.

Welcome firsttimeuser < **ARCULUS GROUND CONTROL CLIENT** Zero Trust Mode: Do not enforce Zero Trust Security

- Dashboard
- Account
- Manage Users
- Manage Edge Devices
- Add Nodes
- Manage Policies
- Plan Missions
- Mission Executions
- Deploy Honeypots
- Attack Metrics
- Blacklist
- About Arculus
- Sign Out

About Arculus

Arculus is a cutting-edge project being developed by the **Cyber Education, Research and Infrastructure (CERi) Center at the University of Missouri - Columbia**. It represents a significant advancement in cyber security solutions, focusing on a **Low Overhead Zero Trust Solution in a Tactical Edge Network setting**. Key features of Arculus include **Microsegmentation** and **Active Defense using Honeypots**. The Microsegmentation approach effectively prevents lateral movement of threat agents.

4. The “Manage Users” tab of the Arculus sidebar takes the user to the “User Management Dashboard” where (s)he (only mission creators) can view existing users, create new users, configure and edit their roles and domains, or delete the users. For a demo purpose, we created a user named “testuser” configured with a “Mission Supervisor” role.

Welcome firsttimeuser < **ARCULUS GROUND CONTROL CLIENT** Zero Trust Mode: Enforce Risk-based Zero Trust Security

- Dashboard
- Account
- Manage Users
- Manage Edge Devices
- Add Nodes
- Manage Policies
- Plan Missions

User Management Dashboard

User Name	E-mail ID	Role	Domains	Action	Action
firsttimeuser	firsttimeuser@missouri.edu	Mission Creator	Explosive Ordnance Reconnaissance Covert Operations Support Structural Integrity Assessment Crisis Response and Restoration		

+ ADD USER

Welcome firsttimeuser < **ARCULUS GROUND CONTROL CLIENT** **Zero Trust Mode:** Enforce Risk-based Zero Trust Security

User Management Dashboard

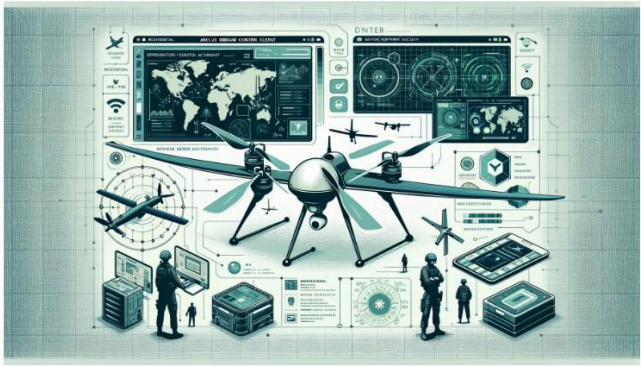
User Name	E-mail ID	Role	Domains	Action	Action
firsttimeuser	firsttimeuser@missouri.edu	Mission Creator	Explosive Ordnance Reconnaissance Covert Operations Support Structural Integrity Assessment Crisis Response and Restoration		
testuser	tester@gmail.com	Mission Supervisor	Covert Operations Support	EDIT	DELETE

[+ ADD USER](#)

5. On signing in as testuser (a Mission Supervisor), we can observe that there are limited dashboard(s) he can navigate to, demonstrating a robust Role-based Access Control for users' interaction with the UI.

Welcome testuser < **ARCULUS GROUND CONTROL CLIENT** **Zero Trust Mode:** Enforce Risk-based Zero Trust Security

[Dashboard](#) [Account](#) [Add Nodes](#) [Manage Policies](#) [Mission Executions](#) [Attack Metrics](#) [Blacklist](#) [About Arculus](#) [Sign Out](#)



About Arculus

Arculus is a cutting-edge project being developed by the **Cyber Education, Research and Infrastructure (CERI) Center at the University of Missouri - Columbia**. It represents a significant advancement in cyber security solutions, focusing on a **Low Overhead Zero Trust Solution in a Tactical Edge Network setting**. Key features of Arculus include **Microsegmentation** and **Active Defense using Honeyhunts**. The Microsegmentation approach effectively prevents lateral movement of threat agents.

6. The more important functionality of device management begins with the “Add Nodes” tab of the sidebar. This page generates custom scripts for new devices to join the Arculus device cluster. On entering the name we want to give the device that joins the cluster, a script is created with the parameter. This script can be copied to clipboard and be run on the device we want to add to the cluster.

Welcome firsttimeuser < **ARCULUS GROUND CONTROL CLIE...** **Zero Trust Mode:** Enforce Risk-based Zero Trust Security

Download both the Arculus Join Request Wizard and the Honeypot Wizard here.

[ARCULUS JOIN REQUEST WIZARD](#)

[ARCULUS HONEYPOT WIZARD](#)

Or pull the scripts through CLI to install.

Enter the node name you want to join to the cluster with (Would not work without this):
surveillancedrone

Join through CLI

```
wait
chmod +x joinClusterWizard.sh
chmod +x honeypotWiz.py
./joinClusterWizard.sh 52.23.225.56 172.31.26.207 surveillancedrone
```

Copied to Clipboard

7. The automated script puts in a join request to the Arculus backend. This request pops up on the Device Management Dashboard of the Arculus UI under the “Cluster Join Requests” section.

```
ubuntu@ip-172-31-47-238:~$ sudo su
root@ip-172-31-47-238:/home/ubuntu# curl -o joinClusterWizard.sh http://52.23.225.56:3001/tools/downloadJoinWiz & curl -o honeypotWiz.py http://52.23.225.56:3001/tools/downloadHoneyWiz &
wait
chmod +x joinClusterWizard.sh
chmod +x honeypotWiz.py
./joinClusterWizard.sh 52.23.225.56 172.31.26.207 surveillancedrone
```

```
0 upgraded, 0 newly installed, 0 to remove and 26 not upgraded.
Cluster join request successful.
Opening websocket to get join status...
ws://52.23.225.56:3002/getJoinStatus?nodeName=surveillancedrone
Received message: Request submitted. Please hang on until an administrator approves your request.
```

Welcome firsttimeuser < **ARCULUS GROUND CONT...** **Zero Trust Mode:** Enforce Risk-based

Device Management Dashboard

Cluster Join Requests	Configured Devices
? 34.230.83.210	More Devices in the Cluster controller

8. Super users (Mission Creators) can view these requests and choose to either approve or decline the request. On approval, the K3s token of the Arculus backend is securely shared with the new device, with which is joins the cluster.

×

Requesting Device Information

Name	IP Address
surveillancedrone	34.230.83.210
✓ APPROVE ✗ DECLINE	

```

Received message: Request submitted. Please hang on until an administrator approves your request.
Join Successful. Proceeding further.
Token received: K102cbcaebf4feecbe1860cf21abe1a5e3d4fba215fa092b3684df4724f8eb56e22::server:12e8262b81ea34d0
sudo: /usr/local/bin/k3s-agent-uninstall.sh: command not found
Failed to stop k3s.service: Unit k3s.service not loaded.
Failed to disable unit: Unit file k3s.service does not exist.
[INFO] Finding release for channel stable
[INFO] Using v1.29.6+k3s2 as release
[INFO] Downloading hash https://github.com/k3s-io/k3s/releases/download/v1.29.6+k3s2/sha256sum-amd64
[INFO] Downloading binary https://github.com/k3s-io/k3s/releases/download/v1.29.6+k3s2/k3s
[INFO] Verifying binary download
[INFO] Installing k3s to /usr/local/bin/k3s
[INFO] Skipping installation of SELinux RPM
[INFO] Creating /usr/local/bin/kubectll symlink to k3s
[INFO] Creating /usr/local/bin/crictll symlink to k3s
[INFO] Creating /usr/local/bin/ctr symlink to k3s
[INFO] Creating killall script /usr/local/bin/k3s-killall.sh
[INFO] Creating uninstall script /usr/local/bin/k3s-agent-uninstall.sh
[INFO] env: Creating environment file /etc/systemd/system/k3s-agent.service.env
[INFO] systemd: Creating service file /etc/systemd/system/k3s-agent.service
[INFO] systemd: Enabling k3s-agent unit
Created symlink /etc/systemd/system/multi-user.target.wants/k3s-agent.service → /etc/systemd/system/k3s-agent.service
[INFO] systemd: Starting k3s-agent

```

- Once the new device sets up all the required dependencies on itself and joins the cluster, it shows up on the Device Management Dashboard under the “More Devices in the Cluster” section.

Welcome firsttimeuser <

ARCULUS GROUND CONT...

▶

Zero Trust Mode: Enforce Risk-based

0

1

Dashboard

Account

Manage Users

Manage Edge Devices

Add Nodes

Manage Policies

Plan Missions

Mission Executions

Deploy Honeyspots

Cluster Join Requests

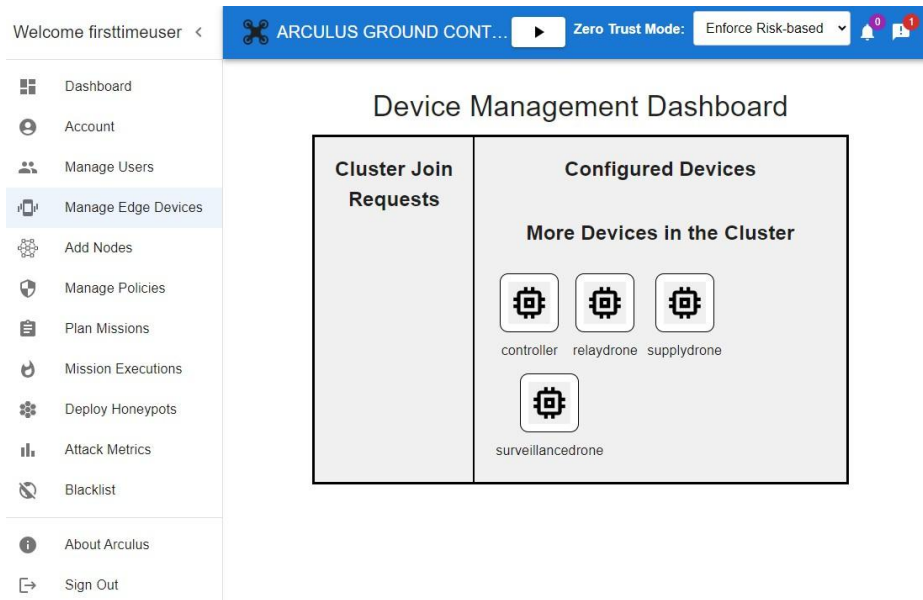
Configured Devices

More Devices in the Cluster

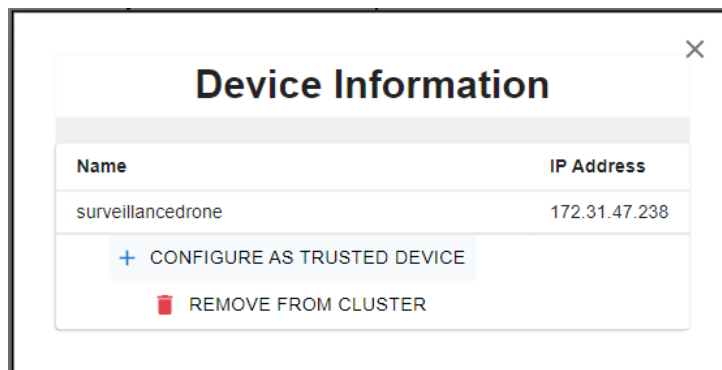
controller

surveillancedrone

- As we would need a surveillance drone, a supply drone, a relay drone and a controller, add two more devices to the cluster using the same procedure as above.



11. On clicking on an device from the “More Devices” section, we can configure them further as trusted devices to attain virtualized behavior by deploying containers on them or remove the devices from the cluster.



12. By configuring the devices as trusted devices, we can deploy them as certain types of drones/rovers/devices with respective functionalities, and adjust the allowed operations for the devices, which dictate the ingress and egress rules for the devices.

Add Trusted Device

Device Name *
surveillancedrone

IP Address *
172.31.47.238

The device will be assigned a local IP address after setup and will no longer be referenced by the public IP address.

Device Type *
Video Capture Drone

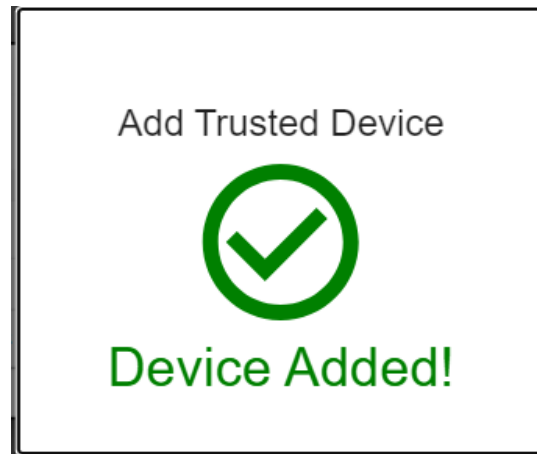
Allowed Operations

send_video
send_sensordata

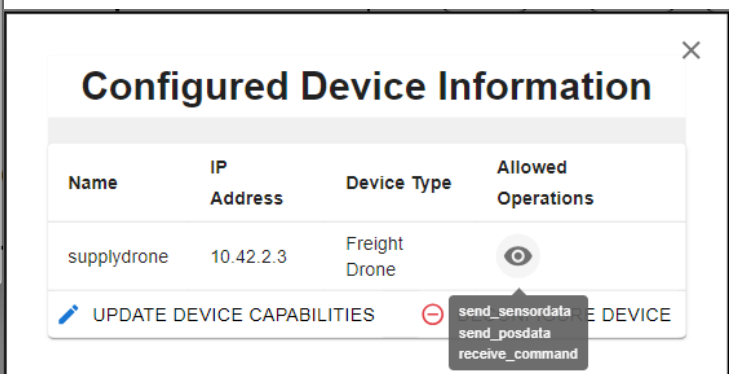
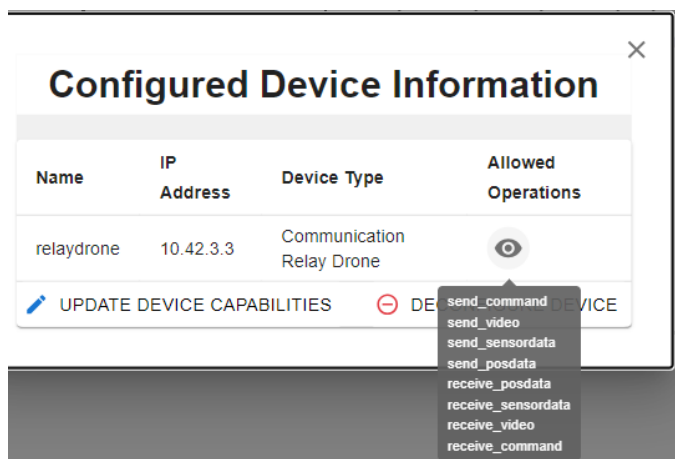
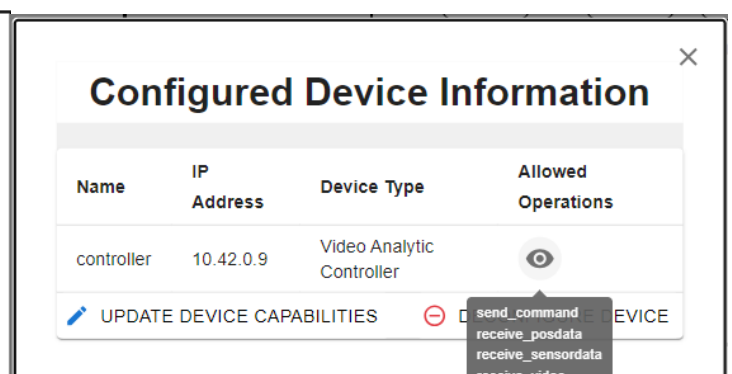
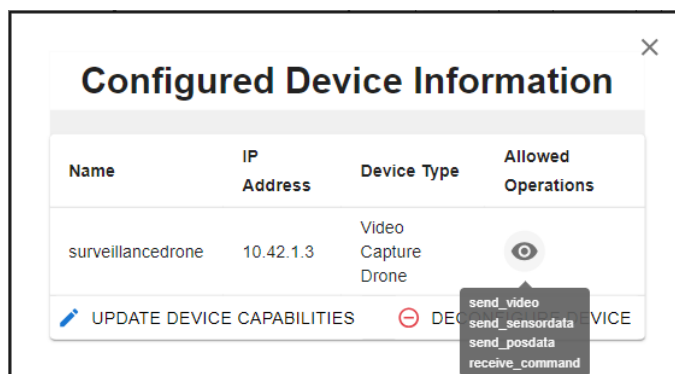
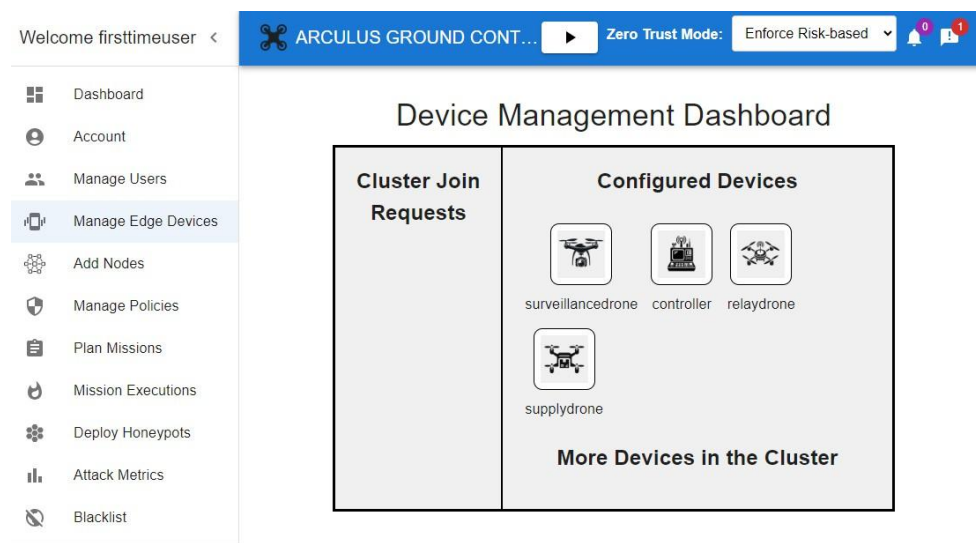
receive_command

Possible Ingress Rules *
5025/TCP

Egress Rules *
5005/UDP
5035/TCP



13. Configure all four devices required for the mission execution with their respective drone types and capabilities as shown in the configuration screenshots below.



IV. Deploying, Managing and Monitoring Honeybots

1. The Honeybot dashboard of the Arculus web interface allows for the seamless deployment of multiple types of honeybots. The target device and the honeybot can be selected based on the type of vulnerabilities to be simulated for active defense using the capabilities of the CHN Server.

The screenshot shows the 'Honeybot Dashboard' in the Arculus web interface. The left sidebar contains navigation links: Dashboard, Account, Manage Users, Manage Edge Devices, Add Nodes, Manage Policies, Plan Missions, Mission Executions, Deploy Honeybots (highlighted), Attack Metrics, and Blacklist. The main content area has a top bar with 'Welcome firsttimeuser', 'ARCULUS GROUND CONTROL CLI...', 'Zero Trust Mode: Enforce Risk-based Zero Trust Secur...', and notification icons. Below the title 'Honeybot Dashboard', there are two tabs: 'Deployed HoneyPots' and 'Deploy HoneyPots'. The 'Deploy New Honeybot' form includes a 'Device IP Address' field with the value '172.31.45.90 (relaydrone)', a 'Honeybot Type' dropdown menu set to 'Cowrie', a 'Honeybot Description' text area with details about the Cowrie bot's intensity and behavior, and a 'Trap Port Mapping' field showing '2222 : 2222, 23 : 2223'. A 'DEPLOY HONEYBOT' button is at the bottom. The footer indicates 'Powered by CommunityHoneyNet (CHN)'.

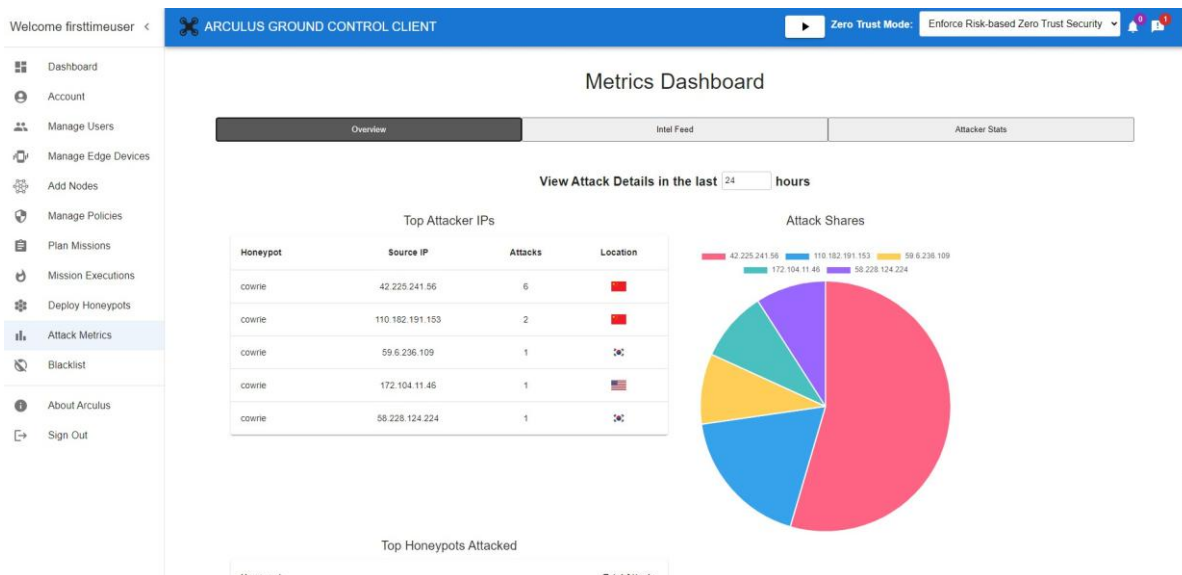
2. The deployed honeybots can be viewed or deleted on the “Deployed Honeybots” pf the Honeybot dashboard.

The screenshot shows the 'Honeybot Dashboard' with the 'Deployed HoneyPots' tab selected. The table below lists the deployed honeybots.

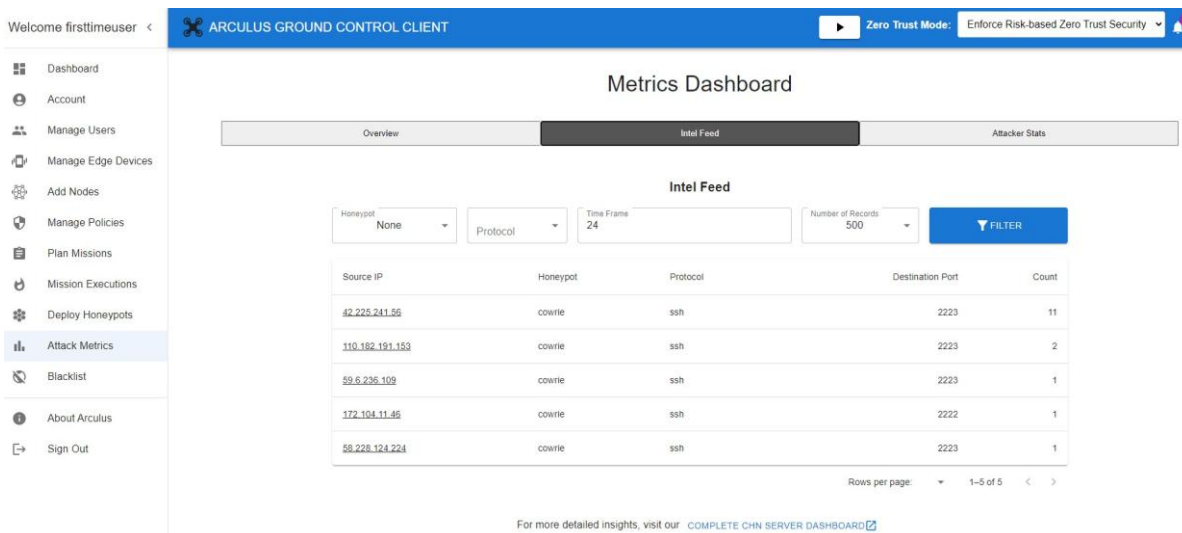
IP Address	Honeybot Type	Actions
172.31.45.90	Cowrie	View Delete

The footer indicates 'Powered by CommunityHoneyNet (CHN)'.

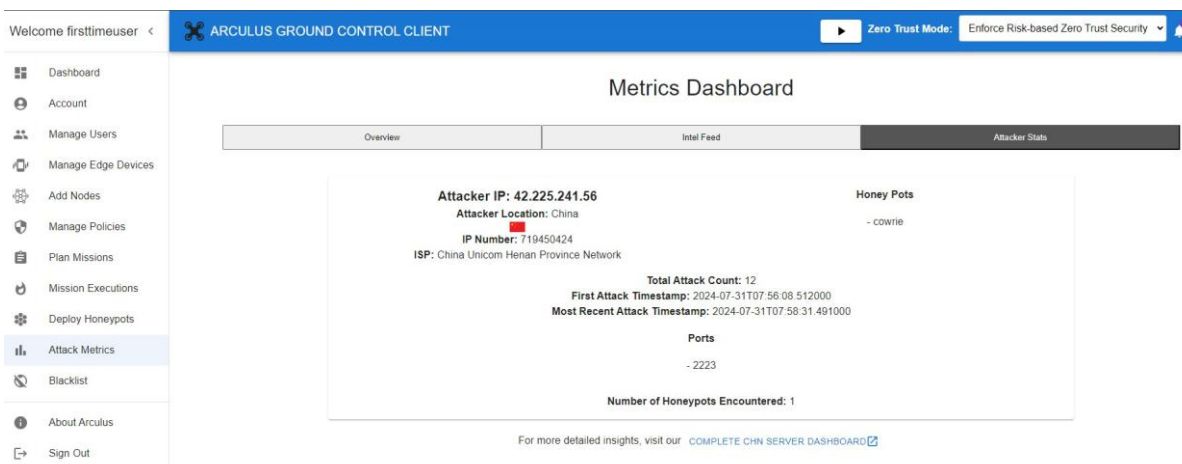
3. On the Metrics Dashboard, we can see the overview of all the attackers and their share of the total malicious activity on our sensors.



- The Intel Feed gives deep and technical insights into what types of attacks using which protocols and ports were executed on the cluster. This gives us information on the strategies attackers are taking as a pattern to establish attack signatures.



- We can also view the “Attacker Stats” tab where a detailed information of the source of attacks and the total count of the attack attempts can be seen. These dashboards provide a lot of threat intelligence which is also backed up on the CHN Server for detailed analysis.



V. Planning and Execution of Drone Missions on the Arculus Testbed

1. The Mission Planning Dashboard allows for configuring drone missions by specifying which devices to use, what the context (criticality) of the mission is, who the supervisors and viewers of the mission are, and logistics information. Configure these options to tune the criticality of the mission dictating the zero trust levels. The device fields are automatically populated based on the availability of the suitable types of devices.

Welcome firsttimeuser < **ARCULUS GROUND CONTROL CLIENT** Zero Trust Mode: Enforce Risk-based Zero Trust Security

Mission Planning Dashboard

Select Mission Type Plan Mission

Plan Mission

Selected Mission Type: Stealthy Reconnaissance and Resupply

Mission Location: Battlefield 1: Paieto Forest **Video-Analytic Route Planner (Ground Control):** controller

Video Collection Surveillance Drone: surveillancedrone **Supply Delivery Drone:** supplydrone

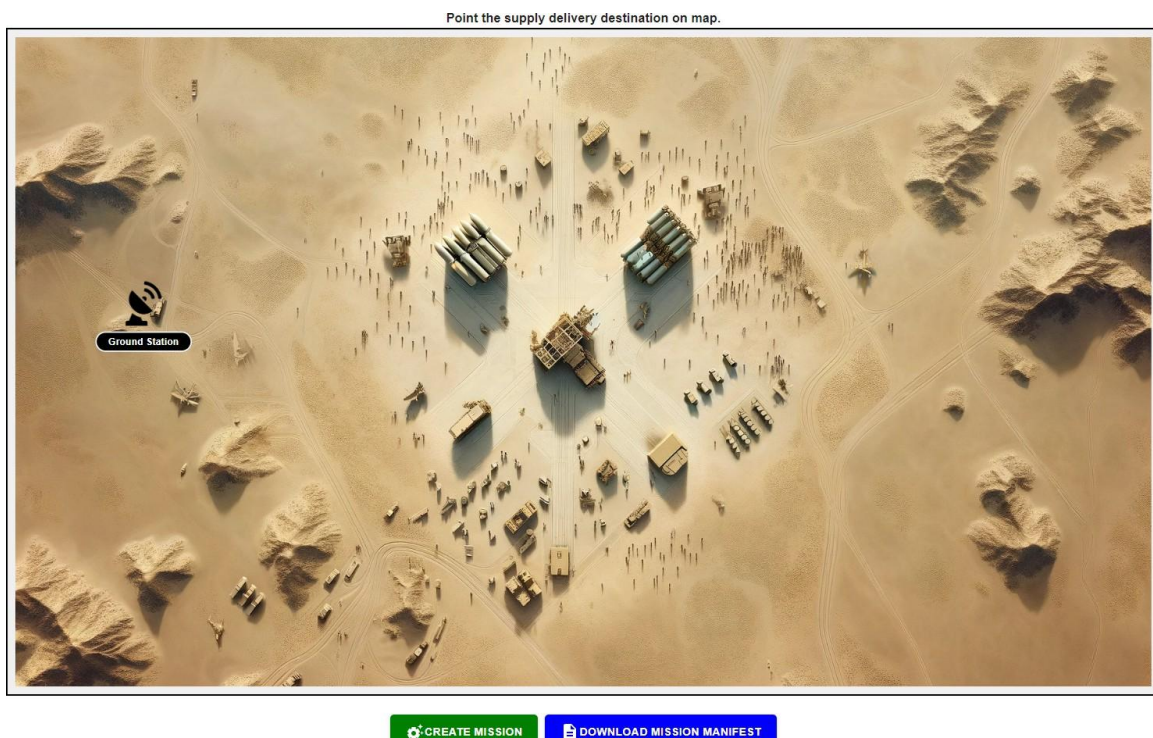
Communication Relay Drone: relaydrone **Asset Criticality:** Low

Life Threat: Low **Data Sensitivity:** Low

Strategic Importance: Low **Supervisors:** testuser

Viewers:

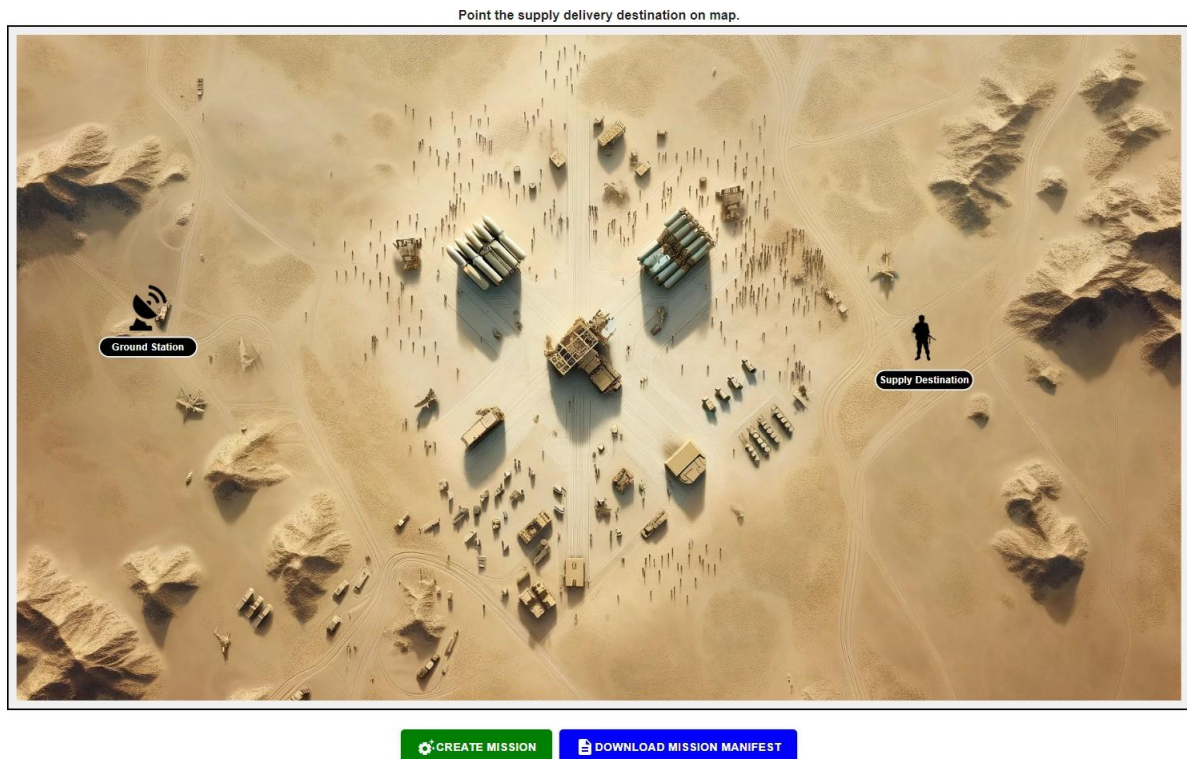
2. The page also provides maps of locations to choose from. On these maps, users can pick a destination point for the delivery of supplies (here, the case of Stealthy Reconnaissance and Resupply mission). Try creating the mission without pointing a destination on the map, and the mission creation will not go ahead asking the user to place a destination on the map.



Please point the destination on the map!

OK

- Click anywhere on the map to set the supply destination to that location.



- Try updating the capabilities and remove one of the privileges specified in the device configuration step. The mission creation won't go ahead because, inability of any of the devices to perform their respective functions will cause the mission to fail for sure. This validation mechanism is a proactive approach to missions whose failure cannot be afforded.

Edit Trusted Device

Device Name *
surveillancedrone

IP Address *
10.42.1.3

Allowed Operations

send_video × send_sensordata ×

receive_command × send_posdata ×

Possible Ingress Rules *

5005/UDP
5015/TCP
5035/TCP

Egress Rules *

5025/TCP

UPDATE TRUSTED DEVICE

CANCEL

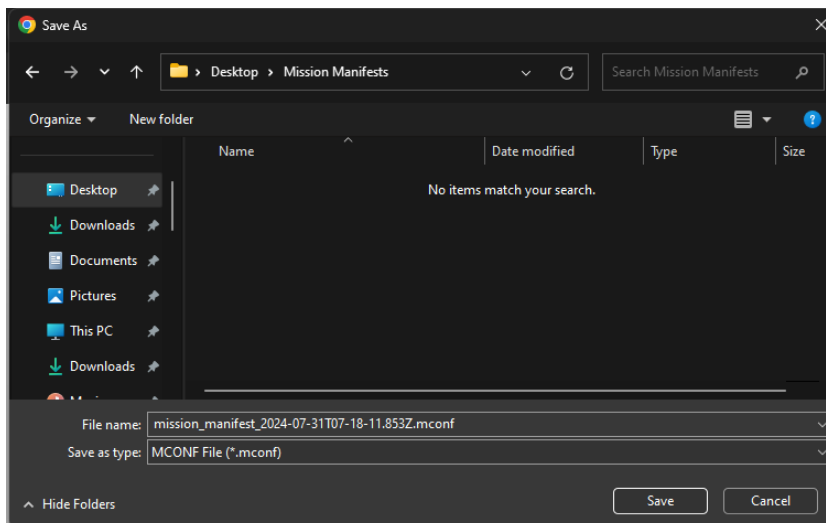
Insufficient Privileges for Some Devices

Mission Execution might fail! Please provide sufficient capabilities for necessary ingress and egress.

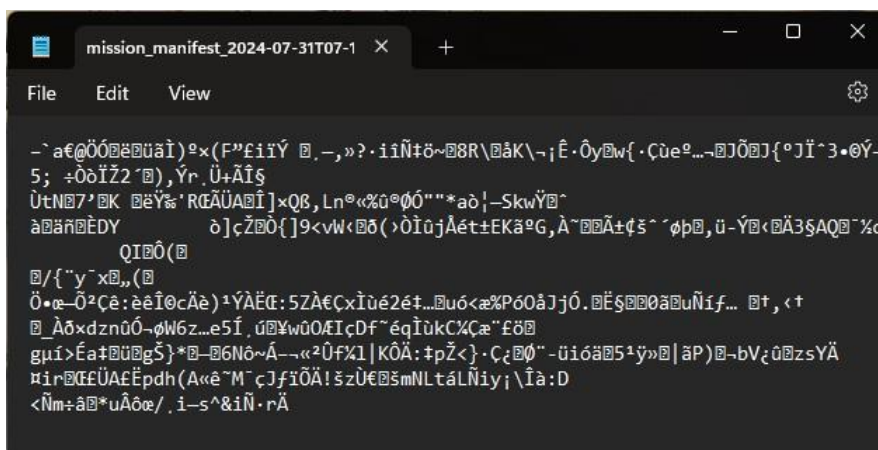
Device	Privileges
controller	☒ send_command ☒ receive_video ☒ receive_posdata
surveillancedrone	☒ send_video ☒ receive_command ☒ send_posdata
supplydrone	☒ receive_command ☒ send_posdata
relaydrone	☒ receive_command ☒ send_command ☒ send_video ☒ send_posdata ☒ receive_posdata ☒ receive_video

OK

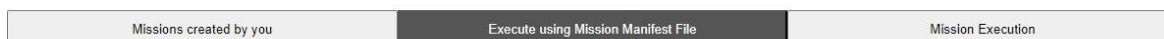
5. Besides normal creation of missions, the “Download Mission Manifest” button allows users to download a repeatable manifest of their mission. This prevents multiple reconfigurations of a mission. The files are encrypted and downloaded with a .mconf extension.



6. From the screenshot below, the manifest files are encrypted and hence, modifications to these files by external actors invalidates the files, leaving no space for deliberate mission adulteration. The next screenshot shows how the Arculus application fetches the mission information from the .mconf file on executing with manifest.



Mission Execution Dashboard

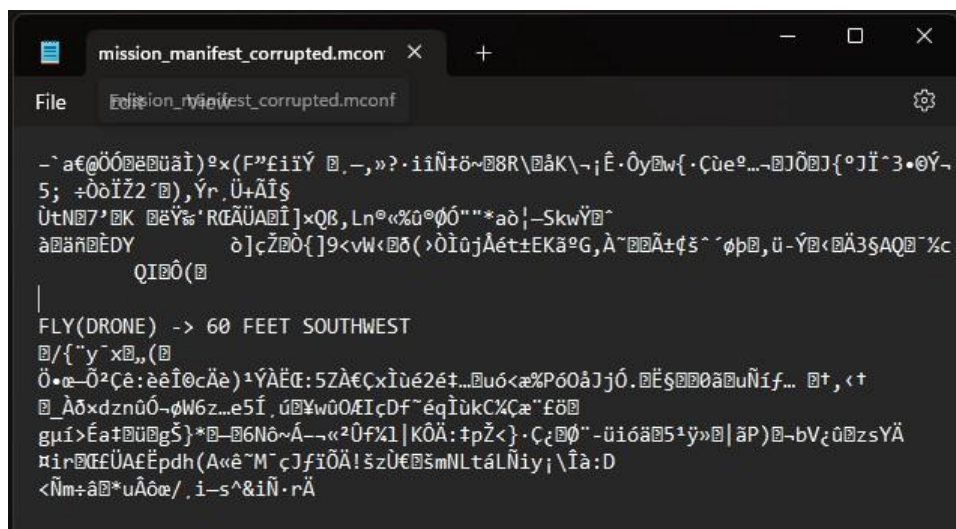


Upload Manifest File for Mission Execution



EXECUTE MISSION

7. However, try adding your own text to the manifest file and try to poison it as shown below. The application throws an error message identifying the manifest file to be bad.



```
-`a€@ÖÖÖüüäi)°x(F"fiïY  .-,»?·iiñtö~8R\âK\¬;Ê·ÖyW{·Çüe°...-ÖJÖJ{°JÏ^3·ÖÝ-
5; ÷òïÏ22  ),Yr.Ü+ÄÏ$
ÛtN7'K  ðèY%·RÆÄÜA  ]xQ8,Ln°«%û°ÖÓ""*aò!-SkwY  ^
àðäñ  EÐY      ò]çŽ  Ö{J9<vW<  Ö( >òIûjÄét±EKâ°G,À~  ÖÄ±çs~`øp ,ü-Ý  <  Ä3$AQ  °`%c
      QIÖÖ(
|
FLY(DRONE) -> 60 FEET SOUTHWEST
  /{"y`x  ,,(
Ö•æ-Ö²Çè:àèI  cÄè)¹ÝÄË:5ZÀ€ÇxÌùé2é±...  uó<æ%P60âJjÓ.  È$  ÖÖÖüüäiñf...  Ö†,<†
  Äð<dzñûÖ-øW6z...e5Í.  ÖYwûÖÆIçDf~éqIùkC%Çæ"Éö  Ö
guí>Éa†ü  g$}*  Ö-Ö6Nô~Ä--«²Üf%1|KÖÄ:†pŽ<}·Ç;  ÖÖ"-üiöä  Ö5¹ÿ»  Ö|äP)  Ö-bVç  ü  ÖzsYÄ
ñir  ÖÆfÜAÉÉpdh(A«è~M`çJfïÖÄ!szÜ€  ÖsmNLtáLÑiy; \Iä:D
<Ñm±â  Ö*uÂôæ/,i-s^&iñ·rÄ
```

Mission Execution Dashboard

Missions created by you

Execute using Mission Manifest File

Mission Execution

Upload Manifest File for Mission Execution

SELECT A DIFFERENT MANIFEST FILE

mission_manifest_corrupted.mconf X

UPLOAD FILE

Bad File! Please choose an actual .mconf file.

Mission Configuration:
Waiting for file upload...

EXECUTE MISSION

8. Before trying to execute a mission, click the “Manage Policies” tab that takes you to the Network Policy Management Dashboard. It can be seen as in the below screenshot, that there are no active network policies and all traffic is denied by default.

ARCULUS GROUND CONTROL CLIENT

Zero Trust Mode: Enforce Risk-based Zero Trust Security

Network Policy Management Dashboard

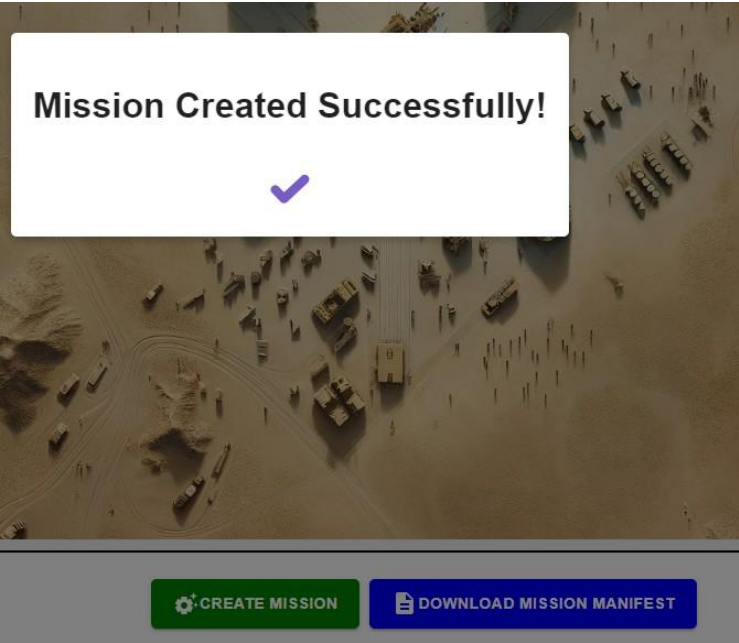
Active Network Policies

New Network Policy



No Active Network Policies (Deny By Default)

9. Now, let us create a mission on the mission planning dashboard and execute it using the EXECUTE (▶) button on the “Missions created by me” tab. The mission starts executing within seconds and the application takes us to the “Mission Execution” tab where the live mission and its logs can be monitored.



Mission Execution Dashboard

Missions created by you			Execute using Mission Manifest File			Mission Execution				
Mission Location	Mission Type	Creator	Supervisors	Viewers	Creation Time	Duration	Execute	Monitor	Status	Delete
Grand Senora Desert	Stealthy Reconnaissance and Resupply	User admin			2024-07-31 07:19:27	120 sec	▶	👁	CREATED	🗑

Mission Execution Dashboard

Missions created by you

Execute using Mission Manifest File

Mission Execution

Simulate Communication Loss | Simulate GPS Spoofing | Simulate Physical Capture | Simulate Denial of Service | Simulate Low Battery | Simulate Brute Force SSH

Ground Control Station: 202.00, 420.00
Surveillance Drone: 472.31, 345.87
Supply Drone: 202.52, 420.04
Destination: 1360.74, 96.30
Status: In Progress (Mission)

Supply Destination

Surveillance Drone

Ground Station

Activity Log

Control Remotely

Surveillance Drone

⬅ ⬆ ⬇ ⬅ ⬆ ⬇ ⬅ ⬆ ⬇

Abort Mission

10. It can be observer that, on navigating to the Network Policy Management Dashboard, new policies are created during the mission execution to facilitate communication for respective tasks. However, as soon as the mission execution finishes/terminates, all the network policies are deleted. This demonstrates the extraordinary and dynamic execution of Task-based Access Control (TBAC) for good privilege management for devices on the network.

Network Policy Management Dashboard

Active Network Policies

New Network Policy

Active Network Policies

Policy ID	Device	Ingress Rules	Egress Rules	Edit	Delete
policy-controller	controller	supplydrone: 6002/UDP surveillancedrone: 6001/UDP relaydrone: 6003/UDP	relaydrone: 5050/TCP surveillancedrone: 3050/TCP supplydrone: 4050/TCP		
policy-relaydrone	relaydrone	controller: 5050/TCP	controller: 6003/UDP		
policy-supplydrone	supplydrone	controller: 4050/TCP relaydrone: 4050/TCP	controller: 6002/UDP		
policy-surveillancedrone	surveillancedrone	controller: 3050/TCP relaydrone: 3050/TCP	controller: 6001/UDP		

11. By playing around with the simulation buttons provided on the Mission Execution Dashboard, we can explore the various zero trust capabilities and countermeasures taken by Arculus for secure and successful completion of missions under adverse conditions. These include DDIL scenarios like communication loss, GPS Spoofing attacks, physical hijacking, Denial of Service, Low battery, Brute force attacks, etc.

Mission Execution Dashboard

Missions created by youExecute using Mission Manifest FileMission Execution

Simulate Communication LossSimulate GPS SpoofingSimulate Physical CaptureSimulate Denial of ServiceSimulate Low BatterySimulate Brute Force SSH

Activity Log

Control Remotely

Abort Mission

Mission Execution Dashboard

Missions created by youExecute using Mission Manifest FileMission Execution

Simulate Communication LossSimulate GPS SpoofingSimulate Physical CaptureSimulate Denial of ServiceSimulate Low BatterySimulate Brute Force SSH

Activity Log

Control Remotely

Abort Mission

VI. Dynamic Zero Trust Grade Updates and Authentication Adaptation

1. The Arculus system continuously evaluates drone risk factors through a Bayesian risk assessment model that automatically updates Zero Trust (ZT) grades based on real-time security conditions and device behavior.
2. Risk assessment factors include attack type detection (particularly physical capture attempts), environmental conditions, behavioral anomalies, network security events, and device health metrics such as battery levels and connectivity status.
3. The ZT grade classification system operates on three levels: Grade A (high risk, bayesian_risk > 85) triggers certificate-based authentication, Grade B (medium risk, 70-85) maintains token-based authentication, and Grade C (low risk, < 70) uses basic authentication mechanisms.
4. When security events are detected, the system immediately recalculates the Bayesian risk score and automatically updates the ZT grade, triggering an immediate change in authentication requirements without manual intervention.

Before ZT grade update:

risk_id	device_id	drone_name	bayesian_risk	attack_type	zero_trust_metric	zt_grade	auth_type	assessed_at	remarks
954	96	drone1	78.61	Normal	78.61	B	AUTH_TOKEN_BASED	2025-09-05 16:31:43	TR=8 Mbps
955	97	drone2	78.61	Normal	78.61	B	AUTH_TOKEN_BASED	2025-09-05 16:31:43	TR=8 Mbps
956	98	drone3	78.61	Normal	78.61	B	AUTH_TOKEN_BASED	2025-09-05 16:31:43	TR=8 Mbps
957	102	droneurv	78.61	Normal	78.61	B	AUTH_TOKEN_BASED	2025-09-05 16:31:43	TR=8 Mbps

After ZT grade update:

risk_id	device_id	drone_name	bayesian_risk	attack_type	zero_trust_metric	zt_grade	auth_type	assessed_at	remarks
958	96	drone1	87.42	Physical Ca...	87.42	A	AUTH_CERT_BASED	2025-09-05 16:32:50	TR=10.4 M
959	97	drone2	84.90	Physical Ca...	84.90	B	AUTH_TOKEN_BASED	2025-09-05 16:32:50	TR=13.5 M
960	98	drone3	87.42	Physical Ca...	87.42	A	AUTH_CERT_BASED	2025-09-05 16:32:50	TR=10.4 M
961	102	droneurv	84.90	Physical Ca...	84.90	B	AUTH_TOKEN_BASED	2025-09-05 16:32:50	TR=12.1 M

5. Authentication method transitions occur in real-time, with the system revoking current tokens, deploying new authentication requirements, and notifying devices of updated security protocols based on the new ZT grade assignment.
6. When the system detects higher risk levels, it automatically strengthens how drones prove their identity: drones with very low risk (grades D and E) don't need special authentication, medium-risk drones (grades B and C) must use secure tokens to verify themselves, and high-risk drones (grade A) require digital certificates and stronger security protocols to access the network.
7. As soon as a drone's risk level changes, the system immediately updates how that drone connects to the network by checking its new grade, writing the appropriate authentication requirement to a file on the drone, and adjusting what network resources the drone can access based on its new security level.
8. The system keeps track of all these security changes through dashboards that show which authentication method each drone is currently using, maintains records of when and why authentication levels changed, and helps operators understand how security events led to stronger or weaker authentication requirements for better decision-making and security analysis.

VII. Reinforcement Learning Module for Adaptive Drone

The Arculus platform integrates a Reinforcement Learning (RL) module, **Arculus-RL**, to provide secure, adaptive path planning for drones operating in contested Tactical Wireless Edge (TWE) environments. The RL agents complement the Zero Trust (ZT) Manager by autonomously making navigation decisions that are policy-compliant even when network connectivity is degraded.

Step 1: Environment Setup

When a mission is initiated with RL enabled, the system generates a grid-based environment with:

- Static obstacles (black cells).
- Dynamic congestion zones (orange cells) that appear and disappear randomly with limited duration.
- Separate start positions for each drone (blue markers).
- Mission goals (green markers) that must be reached without violating ZT constraints.

Each drone maintains a **belief grid** based on its field-of-view (FoV). Areas outside the FoV are initially untrusted, and drones cannot assume their contents until observed. This aligns with the ZT principle of “*never trust, always verify.*”

Step 2: Observation Space Construction

For each simulation step, the RL agent receives a structured observation containing:

- Drone’s own coordinates.
- Goal coordinates and relative direction vector.
- Local 11x11 grid window centered on the drone, showing obstacles, congestion, and heuristic A* guidance.
- The relative position of the other drone (to enforce separation).

Step 3: Decision-Making and Action Selection

Each RL agent selects an action from the set {Up, Down, Left, Right, Hold}.

- Actions are chosen according to the current policy (Q-values).
- During training, actions are selected via **epsilon-greedy exploration**, meaning the agent occasionally tries random moves to learn alternative strategies.
- During deployment, the learned policy is followed greedily to maximize efficiency and safety.

Step 4: Reward Calculation

After each action, the system applies the shaped reward function:

- **+10,000** if the drone reaches its goal.
- **-1** per time step (efficiency pressure).
- **-500** if the drone collides with an obstacle.
- **-20** if the drone enters a congestion zone.
- **-10** if the drone moves adjacent to the other drone.
- **+5** for following the A* heuristic path.
- **-5** for straying too far from the heuristic.
- **+0.5 × distance delta** for each step closer to the goal.

This reward design ensures that drones learn to avoid untrusted regions, minimize risk, and complete missions efficiently.

Step 5: Policy Update and Training Cycle

- Each episode starts with drones at randomized start positions and ends when both goals are reached or a maximum step limit is exceeded.
- RL agents update their Q-networks after each step using observed rewards.
- Training typically runs for **5,000 episodes**, balancing exploration with convergence to a stable navigation policy.
- Learned models are stored as agent_0_trained.pth and agent_1_trained.pth for reuse.

Step 6: Visualization and Outputs

Users can observe RL behavior in two ways:

- **Live Simulation:** A Pygame window displays drones navigating the environment, including FoV outlines, congestion zones, and explored areas.
- **Playback Video:** Milestone episodes are exported as .mp4 videos for demo or review purposes.
- **Logs:** Paths from the last 350 episodes are logged, allowing analysts to evaluate convergence and behavior under different conditions.

This RL module equips Arculus with a **self-learning, Zero Trust-compliant navigation layer**. It ensures that when drones encounter uncertain, adversarial, or DDIL conditions, they can still progress toward mission goals securely and efficiently.

VIII. Federated Learning Intrusion Detection System for Drone Swarms

The following contains the code and experimental artifacts for **FL-IDS**, a hybrid deep learning-based Intrusion Detection System (IDS) integrated with a Federated Learning (FL) framework. The system is designed to detect and mitigate network-level attacks such as DDoS in drone swarms operating in adversarial edge environments like AERPAW.

Overview

The project implements a **hybrid CNN–LSTM–GRU model** trained on a customized dataset combining: The [Edge-IIoT Dataset](#)
Real-world DDoS and SQL injection attack traffic collected on the **AERPAW testbed**
The IDS is trained both in **centralized** and **federated settings**, and evaluated under normal, mildly congested (33%), and severely congested (66%) network scenarios.

Dataset Details

Edge-IIoT Dataset
Protocols: ARP, ICMP, HTTP, TCP, UDP, DNS, MQTT, Modbus
Labels: Binary (Attack/Normal), Multi-class (15+ attack types)
Feature engineering includes label encoding, one-hot encoding, chi-squared feature selection, and scaling.

DNN-EdgelloT-dataset.csv (1.22 GB)



Detail	Compact	Column	10 of 63 columns					
▲ frame.time	▲ ip.src_host		▲ ip.dst_host		▲ arp.dst.proto_ipv4		# arp.opcode	#
2206364 unique values	192.168.0.128	43%	192.168.0.128	43%	0	99%		
	192.168.0.101	30%	192.168.0.101	30%	192.168.0.128	1%		
	Other (589787)	27%	Other (595287)	27%	Other (5552)	0%		
2021 11:44:10.081753000	192.168.0.128		192.168.0.101		0		0.0	0
2021 11:44:10.162218000	192.168.0.101		192.168.0.128		0		0.0	0
2021 11:44:10.162271000	192.168.0.128		192.168.0.101		0		0.0	0
2021 11:44:10.162641000	192.168.0.128		192.168.0.101		0		0.0	0
2021 11:44:10.166132000	192.168.0.101		192.168.0.128		0		0.0	0
2021 11:44:10.166159000	192.168.0.128		192.168.0.101		0		0.0	0

AERPAW Attack Dataset

Captured attacks: DDoS_TCP, DDoS_UDP, DDoS_ICMP, DDoS_HTTP
Preprocessing includes categorical encoding, duplicate removal, and merge with Edge-IIoT.

tcp.ack	tcp.ack_ra	tcp.seq	udp.streat	dns.qry	na	icmp.trans	tcp.dstpor	icmp.seq	icmp.chec	tcp.checks	udp.port	tcp.len	http.conte	dns_1	dns_encoc	tcp.flags	dns_0	http3_0	http1_0	udp.time	http3_1	http1_1	dns.retran	tcp.conne	http1_enc	http2
6	8.96E+08	59	0	0	0	1883	0	0	41098	0	0	0	0	0	0	16	1	0	0	0	0	1	1	0	0	1
5	1.5E+09	56	0	0	0	1883	0	0	23095	0	2	0	0	0	0	25	1	0	0	0	0	1	1	0	0	1
0	0	0	0	0	0	1883	0	0	6193	0	0	0	0	0	0	2	1	0	0	0	0	1	1	0	1	1
565	3.27E+09	1	0	0	0	37574	0	0	33017	0	0	0	1	1	16	0	1	1	0	0	0	0	0	0	0	0
1	3.94E+08	1	0	0	0	9232	0	0	25818	0	0	0	1	1	20	0	0	0	0	0	0	1	1	0	0	1
0	0	6	0	0	0	53634	0	0	18426	0	0	0	0	0	0	4	1	0	0	0	0	1	1	0	0	1
1	2.57E+09	1	0	0	0	1883	0	0	15983	0	14	0	0	0	0	24	1	0	0	0	0	1	1	0	0	1
1	3.66E+09	1	0	0	0	80	0	0	35591	0	427	0	1	1	24	0	0	0	0	0	0	0	0	0	0	2
59	1.64E+09	5	0	0	0	65486	0	0	42193	0	0	0	0	0	0	17	1	0	0	0	0	1	1	0	0	1
523907	2.37E+09	1.39E+08	0	0	0	60944	0	0	60337	0	1440	0	0	0	0	16	1	0	0	0	0	1	1	0	0	1
0	0	442	0	0	0	60186	0	0	617	0	0	0	1	1	4	0	1	1	0	0	0	0	0	0	0	0
0	0	6	0	0	0	60089	0	0	53508	0	0	0	0	0	0	4	1	0	0	0	0	1	1	0	0	1
1	3.12E+09	1	0	0	0	1883	0	0	32892	0	0	0	0	0	0	16	1	0	0	0	0	1	1	0	0	1
0	0	6	0	0	0	60772	0	0	11318	0	0	0	0	0	0	4	1	0	0	0	0	1	1	0	0	1
15	1.82E+09	1	0	0	0	49775	0	0	64755	0	4	0	0	0	0	24	1	0	0	0	0	1	1	0	0	1
0	0	6	0	0	0	51659	0	0	209	0	0	0	0	0	0	4	1	0	0	0	0	1	1	0	0	1
9640	2.8E+08	41234	0	0	0	50104	0	0	49906	0	495	277	1	1	24	0	1	1	0	0	0	0	0	0	0	0
1	3E+09	0	0	0	0	55629	0	0	19153	0	0	0	0	0	0	18	1	0	0	0	0	1	1	0	0	1
487	4.28E+09	1112	0	0	0	80	0	0	606	0	0	0	1	1	16	0	1	1	0	0	0	0	0	0	0	0
6	1.52E+09	59	0	0	0	1883	0	0	47261	0	0	0	0	0	0	16	1	0	0	0	0	1	1	0	0	1
15	4.12E+09	1	0	0	0	54950	0	0	57422	0	4	0	0	0	0	24	1	0	0	0	0	1	1	0	0	1
273197	2.37E+09	71542877	0	0	0	60944	0	0	64518	0	1440	0	0	0	0	16	1	0	0	0	0	1	1	0	0	1
1.62E+08	1.3E+09	602915	0	0	0	5900	0	0	12966	0	0	0	0	0	0	16	1	0	0	0	0	1	1	0	0	1
6	3512829	59	0	0	0	1883	0	0	43623	0	0	0	0	0	0	16	1	0	0	0	0	1	1	0	0	1
0	0	0	0	0	0	0	43688	2126	0	0	0	0	1	1	0	0	0	0	0	0	0	1	1	0	0	1
0	0	0	0	34610	0	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	1	1	0	0	1
1	3.91E+09	0	0	0	0	60512	0	0	11771	0	0	0	0	0	0	18	1	0	0	0	0	1	1	0	0	1
0	0	0	895886	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	1	1	0	0	1
0	0	0	2263548	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	1	1	0	0	1
0	0	0	0	0	0	0	0	0	0	13131	26053	0	0	0	0	1	1	0	0	0	0	1	1	0	0	1
0	0	6	0	0	0	58214	0	0	31431	0	0	0	0	0	0	4	1	0	0	0	0	1	1	0	0	1

Model Architecture

- Conv1D + MaxPooling: Feature extraction
- LSTM + GRU: Captures long- and short-term temporal patterns
- Dense + Dropout: Regularized classification layer
- Configuration:

Parameter	Value
Epochs	6
Batch Size	32
Optimizer	Adam
Loss (Binary)	Binary CrossEntropy
Loss (Multiclass)	Sparse Categorical CrossEntropy
Output Activation	Sigmoid / Softmax
Sequence Shape	(samples, time_steps, features)

Experiments

- Goals:
- Detect and Mitigate DDoS (network-level)
- Evaluate performance under 33% and 66% attack intensities
- Environment:
- Testbed Setup
- Server: A compute node (VM/Laptop machine)
- Clients: Starling VOXL2 drone and Raspberry Pi 3
- Network tools: iperf3, tc

Evaluation Metrics

Metric Type	Metrics
Model	Accuracy, Precision, Recall, F1
System	CPU usage, Training Time
Network	Bandwidth, RTT, Packet Loss, Attack Recovery Time

Results Snapshot

Binary Classification Accuracy (FL): ~0.98
Multiclass Precision/Recall (centralized): ~0.94
DDoS Traffic Detection: >94% accuracy under realistic conditions
Federated Recovery: Maintained accuracy even with 1–3 poisoned nodes

How to Run

Setup and Install dependencies

```
# Clone repository
git clone https://github.com/arculus-zt/arculus-fl
```

```
# Install dependencies
pip install -r requirements.txt
```

Centralized Training

Run centralized model

For centralized binary classification
python3 centralized/binary_classification.py
For centralized multiclass classification
python3 centralized/multiclass_classification.py

Federated Training

Choose the directory which model you wanna train (binary/multiclass)

```
cd federated/binary #for binary classification
or
cd federated/multiclass #for multiclass classification
```

Launch the server

```
python3 server.py
```

Launch federated clients (on different terminals or nodes)

```
python3 federated/client.py -i 1
```

here **i** is the ID of the individual FL participant node client. For the second client, run the same script with -i 2

DDoS Attack Generation

Controlled Intensified DDoS using Iperf3

This command uses iperf3 to measure high-throughput network performance between a client and server. It's configured to simulate a sustained, high-bandwidth data transfer using multiple parallel streams.

On client nodes (destination)

Start the iPerf3 server on the destination machine:
iperf -s -i 1

On the attacker node (source)

```
iperf -c 192.168.129.1 -b 4.8G -P 8 -w 1M -i 1 -t 3000 #{33%}
```

Flag / Option	Description
-c 192.168.129.1	Run in client mode and connect to the iPerf3 server (e.g., 192.168.129.1)
-b 4.8G	Set target bandwidth (e.g., 4.8) with to consume 33%/66% of the total available bandwidth
-P 8	Use 8 parallel connections (streams) for higher throughput
-w 1M	Set TCP window size (socket buffer) to 1 Megabyte
-i 1	Report stats every 1 second during the test
-t 3000	Run the test for 3000 seconds (50 minutes)

Implementation:

There are several steps that you need to take before we implement the procedures. We need to have a terminal in the ground control station (GCS) that is the certain part for connecting the different clients that run the federated learning models (individual local nodes).

Initializing the server node:

```
~/Downloads -- -zsh      ~/Downloads/FL-IDS/federated/binary -- -zsh      ~/Downloads/FL-IDS -- -zsh      ...~/FL-IDS/federated/binary -- ssh pi@192.168.1.199 +
bwbgv@CENGR-KV86WFDH44 Downloads % cd FL-IDS
bwbgv@CENGR-KV86WFDH44 FL-IDS % source venv/bin/activate
(venv) bwbgv@CENGR-KV86WFDH44 FL-IDS % ls
aerpaw65.py          ddos_icmp_attack.csv      final_experiment_tree.drawio      requirements.txt
aerpawfinal.py       ddos_icmp_attack.pcap     fl_training_times_nd33_baseline.pdf  results
app                  ddos_tcp_attack.csv       fl_training_times_nd66_baseline.pdf  results.zip
attack.csv           ddos_tcp_attack.pcap     flypawPilot_first.py              sql_injection_attack_to_merge.csv
attacks              ddos_udp_attack.csv      linegraph.py                       sql_injection_attack.pcap
baseline_shaded.py   ddos_udp_attack.pcap     mermaid.py                         sql_injection.py
baseline.py          diagram.md                newplots                            training_times_synthesized.pdf
centralized          DNN-EdgeIIoT-dataset.csv  dump.csv                           treeexperimentfull.drawio
combined_edgelliot_500k_custom_DDos.csv  dump.pcap                       normal.csv                          updated_figure.png
combined_Normal_UDP_TCP.py  EdgeIIoTset-12-filtered-DDos-TCP.csv  plotideas.py                       validation_edgelliot_500k_custom_DDos.csv
custom_attack.csv      EdgeIIoT_99K_subset.csv   preprocessing                       venv
ddos_http_attack_to_merge.csv  federated                  Preprocessing_misc.ipynb
ddos_http_attack.csv   fields.txt                README.md
ddos_http_attack.pcap
(venv) bwbgv@CENGR-KV86WFDH44 FL-IDS % cd federated
(venv) bwbgv@CENGR-KV86WFDH44 federated % ls
binary                Federated_datasets         multiclass
(venv) bwbgv@CENGR-KV86WFDH44 federated % cd binary
(venv) bwbgv@CENGR-KV86WFDH44 binary % ls
attack_type_distribution.png  client.py                  server.py
(venv) bwbgv@CENGR-KV86WFDH44 binary % python server.py
```

Access the terminal through the GCS and enter the directory containing the FL-IDS. Inside the FL-IDS folder, access the ‘federated’ folder and change directory again to the ‘binary’ folder to gain access to the model that works as a binary classifier.

Run the server.py script that initializes the GCS as the main server that contains the global model for the IDS model.

Initializing the Drone node:

```
~/Downloads -- -zsh      ...nloads/FL-IDS/federated/binary -- Python server.py      ... x      ~/Downloads/FL-IDS -- ssh root@192.168.1.233
((venv) bwbgv@CENGR-KV86WFDH44 FL-IDS % ssh root@192.168.1.233
root@192.168.1.233's password:
Welcome to Ubuntu 18.04.5 LTS (GNU/Linux 4.19.125 aarch64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge
This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Mon Aug  4 17:01:42 2025 from 192.168.1.87

MOON AI

system-image: 1.7.10-M0054-14.1a-perf
kernel:      #1 SMP PREEMPT Fri Sep 27 21:59:02 UTC 2024 4.19.125
-----
hw platform: M0054
mach.var:    1.0.1
-----
voxl-suite:  1.3.5
-----
current IP:  wlan0: 192.168.1.233
-----

voxl2:~$ ls
FL-IDS  eeprom  fastapi  flask  log  readme.txt  vam_test
voxl2:~$ cd FL-IDS/
voxl2:~/FL-IDS$ ls
FL-IDS  venv
voxl2:~/FL-IDS$ source venv/bin/activate
(venv) voxl2:~/FL-IDS$ cd FL-IDS/
(venv) voxl2:~/FL-IDS/FL-IDS(main)$ ls
app  centralized  federated  requirements.txt  results  sql_injection.py  sql_injection_attack.pcap
(venv) voxl2:~/FL-IDS/FL-IDS(main)$ cd federated/binary/
(venv) voxl2:~/FL-IDS/FL-IDS/federated/binary(main)$ python3 client.py -i 1
```

Through the GCS terminal, try ssh into the drone node specifying the IP address to your drone. Follow the same directory path from FL-IDS/federated/binary and initialize the drone as the first client with the code:

Python3 client.py -i 1

Initializing the RPi node:

```

~/Downloads -- zsh
...nloads/FL-IDS/federated/binary -- Python server.py ...
~/Downloads/FL-IDS -- ssh root@192.168.1.233 ...
~/FL-IDS/federated

bwbgv@CENGR-KV06WFDH44 binary % ssh pi@192.168.1.199
pi@192.168.1.199's password:
Permission denied, please try again.
pi@192.168.1.199's password:
Last login: Mon Aug 4 19:12:32 2025 from 192.168.1.87
pi@ubuntu:~$ ls
FL-IDS PX4-Autopilot ros2_foxy ros2_ws rox2_foxy vox1-dev vox12_SDK_1.5.0 vox12_SDK_1.5.0.tar.gz
pi@ubuntu:~$ cd FL-IDS/
pi@ubuntu:~/FL-IDS$ ls
Preprocessing_misc.ipynb README.md baseline.py ddos_http_attack.pcap ddos_udp_attack.pcap final_experiment_tree.drawio preprocessing sql_injection.py
Python-3.10.12 aerpaw66.py baseline_shaded.py ddos_icmp_attack.pcap dump.pcap flypawPilot_first.py requirements.txt sql_injection_attack.pcap
Python-3.10.12.tgz aerpawfinal.py centralized ddos_tcp_attack.pcap federated newplot.py results treeexperimentfull.drawio
pi@ubuntu:~/FL-IDS$ cd federated/
pi@ubuntu:~/FL-IDS/federated$ ls
binary federated_datasets multiclass
pi@ubuntu:~/FL-IDS/federated$ cd binary/
pi@ubuntu:~/FL-IDS/federated/binary$ ls
accuracy_plot_1.jpg accuracy_plot_3.jpg client.py con_max_client2.jpg con_percent_client2.jpg loss_plot_2.jpg server.py
accuracy_plot_2.jpg attack_type_distribution.png con_max_client1.jpg con_percent_client1.jpg loss_plot_1.jpg loss_plot_3.jpg
pi@ubuntu:~/FL-IDS/federated/binary$ python3 client.py -i 2

```

Through the GCS terminal, try ssh into the RPi node specifying the IP address to your RPi. Follow the same directory path from FL-IDS/federated/binary and initialize the drone as the second client with the code in the terminal:

Python3 client.py -i 2

You have established the different nodes with stating them as client 1 and 2.

Model: "sequential"

Layer (type)	Output Shape	Param #
conv1d (Conv1D)	(None, 59, 32)	128
max_pooling1d (MaxPooling1D)	(None, 29, 32)	0
conv1d_1 (Conv1D)	(None, 27, 64)	6,208
max_pooling1d_1 (MaxPooling1D)	(None, 13, 64)	0
lstm (LSTM)	(None, 13, 64)	33,024
gru (GRU)	(None, 64)	24,960
flatten (Flatten)	(None, 64)	0
dense (Dense)	(None, 128)	8,320
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 1)	129

Total params: 72,769 (284.25 KB)
Trainable params: 72,769 (284.25 KB)
Non-trainable params: 0 (0.00 B)
WARNING : DEPRECATED FEATURE: flwr.server.start_server() is deprecated.
Instead, use the `flower-superlink` CLI command to start a SuperLink as shown below:

```
$ flower-superlink --insecure
```

To view usage and all available options, run:

```
$ flower-superlink --help
```

Using `start\_server()` is deprecated.

This is a deprecated feature. It will be removed entirely in future versions of Flower.

```

INFO : Starting Flower server, config: num_rounds=1, no round_timeout
INFO : Flower ECE: gRPC server running (1 rounds), SSL is disabled
INFO : [INIT]
INFO : Requesting initial parameters from one random client
INFO : Received initial parameters from one random client
INFO : Starting evaluation of initial global parameters
INFO : Evaluation returned no results ('None')
INFO :
INFO : [ROUND 1]

```

The global model already consists of the previous parameters and shows information from the last trained session.

```

(venv) vox12:~/FL-IDS/FL-IDS(main)$ cd federated/binary/
(venv) vox12:~/FL-IDS/FL-IDS/federated/binary(main)$ python3 client.py -i 1
Model: "sequential"

```

Layer (type)	Output Shape	Param #
conv1d (Conv1D)	(None, 59, 32)	128
max_pooling1d (MaxPooling1D)	(None, 29, 32)	0
conv1d_1 (Conv1D)	(None, 27, 64)	6,208
max_pooling1d_1 (MaxPooling1D)	(None, 13, 64)	0
lstm (LSTM)	(None, 13, 64)	33,024
gru (GRU)	(None, 64)	24,960
flatten (Flatten)	(None, 64)	0
dense (Dense)	(None, 128)	8,320
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 1)	129

Total params: 72,769 (284.25 KB)
Trainable params: 72,769 (284.25 KB)
Non-trainable params: 0 (0.00 B)
WARNING : DEPRECATED FEATURE: flwr.client.start_client() is deprecated.
 Instead, use the `flower-supernode` CLI command to start a SuperNode as shown below:

```

$ flower-supernode --insecure --superlink='<IP>:<PORT>'

```

To view all available options, run:

```

$ flower-supernode --help

```

Using `start\_client()` is deprecated.

This is a deprecated feature. It will be removed entirely in future versions of Flower.

```

INFO :
INFO : Received: get_parameters message 4c59a033-236f-461f-80c6-a5c51c7bcc56
INFO : Sent reply

```

After the first node has been added the above update it shown with the parameter and the nametag for the client added to it. The confirmation of the client being accepted to the server is shown through “Sent reply” as seen on the image.

Similarly, another message pops up in the terminal accessing the second node, here the RPi, notifying the clients have been successfully added to the GCS. Shown in the image below:

```
[pi@ubuntu:~/FL-IDS/federated/binary$ python3 client.py -i 2
```

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv1d (Conv1D)	(None, 59, 32)	128
max_pooling1d (MaxPooling1D)	(None, 29, 32)	0
conv1d_1 (Conv1D)	(None, 27, 64)	6,208
max_pooling1d_1 (MaxPooling1D)	(None, 13, 64)	0
lstm (LSTM)	(None, 13, 64)	33,024
gru (GRU)	(None, 64)	24,960
flatten (Flatten)	(None, 64)	0
dense (Dense)	(None, 128)	8,320
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 1)	129

Total params: 72,769 (284.25 KB)

Trainable params: 72,769 (284.25 KB)

Non-trainable params: 0 (0.00 B)

WARNING : DEPRECATED FEATURE: flwr.client.start_client() is deprecated.
Instead, use the `flower-supernode` CLI command to start a SuperNode as shown below:

```
$ flower-supernode --insecure --superlink='<IP>:<PORT>'
```

To view all available options, run:

```
$ flower-supernode --help
```

Using `start\_client()` is deprecated.

This is a deprecated feature. It will be removed entirely in future versions of Flower.

The terminal of the GCS shows the following message as soon as both the clients are configured correctly.

```
INFO : Starting Flower server, config: num_rounds=1, no round_timeout
INFO : Flower ECE: gRPC server running (1 rounds), SSL is disabled
INFO : [INIT]
INFO : Requesting initial parameters from one random client
INFO : Received initial parameters from one random client
INFO : Starting evaluation of initial global parameters
INFO : Evaluation returned no results (`None`)
INFO :
INFO : [ROUND 1]
INFO : configure_fit: strategy sampled 2 clients (out of 2)
```

As soon as the clients are confirmed, the training starts on both the client. The outputs look similar on the terminal. The training times are directly related to the resources available in the individual computing resources. Therefore, in this particular experiment, the drone client trains a little slower than the RPI client.

Training:

The training process looks like this:

```
INFO :  
INFO : Received: train message c3920b16-e0f5-4634-81c2-51b45b5cfb40  
Epoch 1/6  
7/711 ————— 43s 61ms/step - accuracy: 0.6098 - loss: 0.6892
```

After the training epochs completes, the local model sends the updates the latest parameters to the server (GCS) that adjusts the global model that is depicted here:

```
INFO :  
INFO : Received: train message c3920b16-e0f5-4634-81c2-51b45b5cfb40  
Epoch 1/6  
711/711 ————— 66s 87ms/step - accuracy: 0.9426 - loss: 0.1630 - val_accuracy: 0.9997 - val_loss: 5.9271e-04  
Epoch 2/6  
711/711 ————— 59s 83ms/step - accuracy: 0.9994 - loss: 0.0017 - val_accuracy: 0.9997 - val_loss: 0.0011  
Epoch 3/6  
711/711 ————— 59s 83ms/step - accuracy: 0.9982 - loss: 0.0058 - val_accuracy: 0.9997 - val_loss: 3.6758e-04  
Epoch 4/6  
711/711 ————— 59s 83ms/step - accuracy: 0.9994 - loss: 0.0011 - val_accuracy: 0.9997 - val_loss: 4.1847e-04  
Epoch 5/6  
711/711 ————— 59s 83ms/step - accuracy: 0.9998 - loss: 5.0111e-04 - val_accuracy: 0.9997 - val_loss: 7.4089e-04  
Epoch 6/6  
711/711 ————— 60s 84ms/step - accuracy: 0.9999 - loss: 5.2180e-04 - val_accuracy: 0.9994 - val_loss: 0.0014  
Training time: 362.02 seconds  
INFO : Sent reply
```

The training times and parameters are updated by both the clients. You can observe the difference in the name tag for both these clients and they send their updated parameters to the global model to make adjustments.

```
INFO :  
INFO : Received: train message 50533c21-eda4-4aba-bdeb-df739d671dd4  
Epoch 1/6  
  
711/711 ————— 63s 69ms/step - accuracy: 0.9177 - loss: 0.1683 - val_accuracy: 0.9997 - val_loss: 0.0016  
Epoch 2/6  
711/711 ————— 48s 67ms/step - accuracy: 0.9998 - loss: 0.0010 - val_accuracy: 0.9991 - val_loss: 0.0030  
Epoch 3/6  
711/711 ————— 48s 67ms/step - accuracy: 0.9994 - loss: 0.0015 - val_accuracy: 0.9997 - val_loss: 0.0013  
Epoch 4/6  
711/711 ————— 48s 67ms/step - accuracy: 0.9999 - loss: 8.1337e-04 - val_accuracy: 0.9997 - val_loss: 0.0044  
Epoch 5/6  
711/711 ————— 48s 67ms/step - accuracy: 0.9998 - loss: 0.0016 - val_accuracy: 0.9997 - val_loss: 0.0011  
Epoch 6/6  
711/711 ————— 48s 67ms/step - accuracy: 0.9999 - loss: 2.0814e-04 - val_accuracy: 0.9997 - val_loss: 4.2978e-04  
Training time: 302.21 seconds  
INFO : Sent reply
```

After training completion, each of the clients performs an evaluation of the model in the remaining data flow for each of the clients.

Evaluation:

After evaluation, the clients request disconnection and shut down.

```
INFO : Received: evaluate message 1ca3a5fe-c5c0-44a2-b4bb-95e341ff49d9  
469/469 ————— 11s 22ms/step - accuracy: 1.0000 - loss: 0.0024  
469/469 ————— 11s 21ms/step  
Testing time: 22.96 seconds  
/home/pi/.local/lib/python3.10/site-packages/sklearn/metrics/_classification.py:407:  
ls' parameter to pass all known labels.  
warnings.warn(  
INFO : Sent reply  
INFO :  
INFO : Received: reconnect message dcc47054-1438-4b4d-92fd-39c812254735  
INFO : Disconnect and shut down
```

The evaluation metrics from the model are reflected in the global model in the GCS server node.

Evaluation Results:

The evaluation metrics such as accuracy, f1-score, precision and recall are reflected to the user. The output looks like this:

```
INFO : Starting Flower server, config: num_rounds=1, no round_timeout
INFO : Flower ECE: gRPC server running (1 rounds), SSL is disabled
INFO : [INIT]
INFO : Requesting initial parameters from one random client
INFO : Received initial parameters from one random client
INFO : Starting evaluation of initial global parameters
INFO : Evaluation returned no results (`None`)
INFO :
INFO : [ROUND 1]
INFO : configure_fit: strategy sampled 2 clients (out of 2)
INFO : aggregate_fit: received 2 results and 0 failures
WARNING : No fit_metrics_aggregation_fn provided
INFO : configure_evaluate: strategy sampled 2 clients (out of 2)
INFO : aggregate_evaluate: received 2 results and 0 failures
[Round 1] Aggregated client metrics:
{'accuracy': 1.0, 'f1_score': 1.0, 'precision': 1.0, 'recall': 1.0}
INFO :
INFO : [SUMMARY]
INFO : Run finished 1 round(s) in 454.23s
INFO : History (loss, distributed):
INFO : round 1: 0.005180048989132047
INFO : History (metrics, distributed, evaluate):
INFO : {'accuracy': [(1, 1.0)],
INFO : 'f1_score': [(1, 1.0)],
INFO : 'precision': [(1, 1.0)],
INFO : 'recall': [(1, 1.0)]}
```
